# *EXPERIMENT 13*

## *CaseStudyontheLimitations of Low-Code Platforms for a Complex Enterprise Use Case*

Global ClaimsandOrder ProcessingEnterprise (Platform-Neutral CaseStudy)

***pavithran.m***

241401065

241401065

# *1. Aim*

To examine, through a case study, the limits that low-code platforms as a category reach when they are applied to a complex enterprise system, rather than judging any single vendor. The scenario is a company that handles customer orders and warranty claims in many countries, and the study ends with suggestions for keeping those limits under control.

The case study asks three things: where low-code clearly helps, where it begins to struggle as scale and complexity increase, and how a hybrid design can keep the speed of low-code while covering its weak points.

# *2. Procedure*

***Step 1: Set the scenario.*** A global company was described that takes orders and claims through web and mobile portals, serves agents, finance staff and administrators, runs a legacy ERP or mainframe, uses payment and logistics partner APIs, and must follow tax, privacy and audit rules that differ by country.

***Step 2: Derive the requirements.*** The scenario was split into six requirement areas: scale, business rules, integration, security and compliance, tailored screens, and long-term upkeep (Section 3.2).

***Step 3: Draw the integration map.*** Every link to the platform was drawn and coloured by how well a typical low-code platform supports it: well supported, possible with effort, or strained (Section 3.1).

***Step 4: Analyse the limits.*** Each common limitation was written down with its effect on this company and a way to reduce it (Section 3.3).

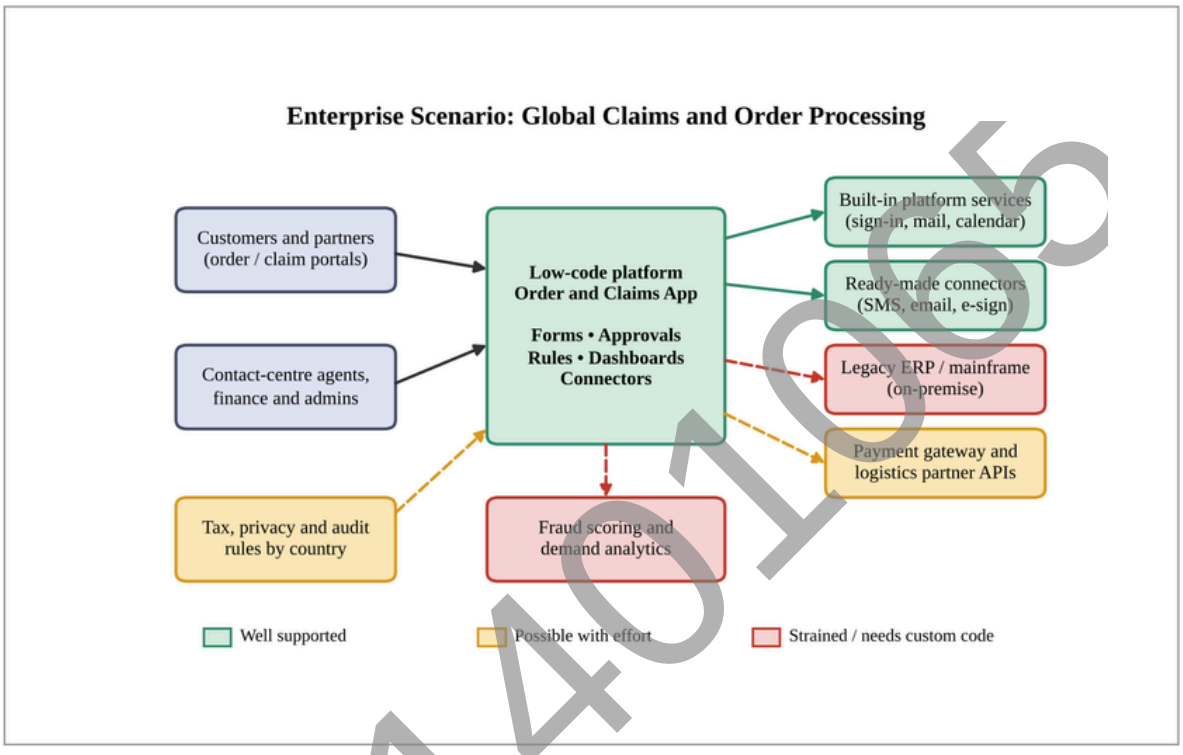
***Step 5: Show the strain.*** A conceptual curve and an indicative suitability chart were prepared to show where low-code fits and where it does not (Sections 3. and 3.5).

***Step 6: Choose and recommend.*** A decision flow was drawn for picking between low-code, hybrid and custom development (Section 3.6), followed by a recommended hybrid architecture (Section 3.7).

***Note: this is an analytical case study. Actual limits such as run time, record counts or API calls vary by platform and plan, so no figures are quoted; the documentation of the chosen platform should be checked. The charts are conceptual or indicative, not measured benchmarks.***

# 3. Output

## 3.1 Enterprise scenario and integration map




The map places a generic low-code platform at the centre of the company. Built-in platform services and ready-made connectors are well supported (green). Payment and logistics APIs, and the country-by-country tax, privacy and audit rules, are possible with effort (amber). The legacy ERP or mainframe, and fraud scoring or demand analytics, are strained and usually need custom code (red).

## 3.2 Enterprise requirements

The requirements drawn from the scenario are listed below.

| *Requirement* | *Why it is complex* |
|---|---|
| Scale | Orders and claims arrive from many regions, with sharp peaks during sales seasons. |
| Business rules | Tax, currency, warranty and claim rules differ by country and change frequently. |
| System integration | A legacy ERP or mainframe, payment and logistics APIs, and company-wide sign-in. |
| Security and compliance | Privacy and financial rules require audit trails, access control and local data storage. |
| Tailored screens | Customers, agents and finance staff each need a different interface and language. |
| Long-term upkeep | Many teams change the system over the years, and the company may switch platform later. |

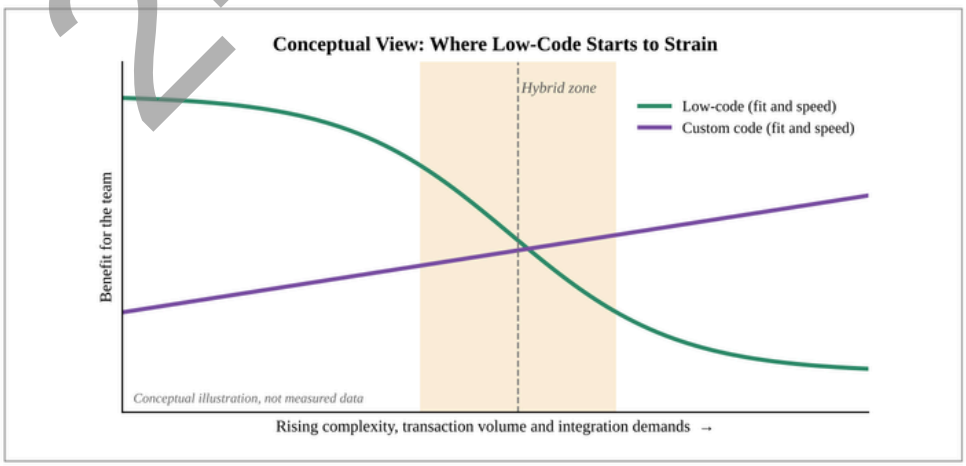
24140105

### *3.3 Limitation analysis*

The limitations below are common across low-code platforms. Each is shown with its effect on this company and a way to reduce it.

| *Area* | *Limitation of low-code* | *Impact on this case* | *Mitigation* |
|---|---|---|---|
| Performance at scale | Shared, vendor-managed runtime with usage caps set by plan. | Peak-season order surges and large reports may slow down or hit caps. | Tune queries, avoid heavy loops, push bulk jobs to external services. |
| Complex logic | Built-in scripting suits routine rules, not advanced algorithms. | Fraud checks and dynamic pricing are hard to express and maintain. | Place the logic in a custom service and call it by API. |
| Integration | Ready connectors cover popular tools; legacy and industry-specific interfaces need extra work. | The ERP, mainframe and partner systems cannot be joined by a simple connector. | Add an API gateway or middleware between the platform and legacy systems. |
| Customisation | Interface follows fixed components and platform conventions. | Highly tailored, multi-language screens are difficult to build. | Use custom components or a separate front end over the platform API. |
| Compliance and data residency | Hosting regions, encryption choices and audit depth depend on the vendor. | Some countries may require more control than the platform gives. | Check vendor certifications, restrict access by role, store sensitive data in a controlled system. |
| Testing and change control | Version history, automated tests and release tools are often basic. | A faulty tax or claim rule change could reach live users unnoticed. | Use test environments, peer review and a written release routine. |
| Sprawl | Easy building lets each team create its own apps. | Duplicate, insecure or unsupported apps appear across regions. | Keep a central app register, shared standards and a named owner per app. |
| Vendor lock-in | Apps and workflows are bound to one platform and hard to export. | A later change of platform means rebuilding from scratch. | Keep core logic in APIs and keep data exportable. |

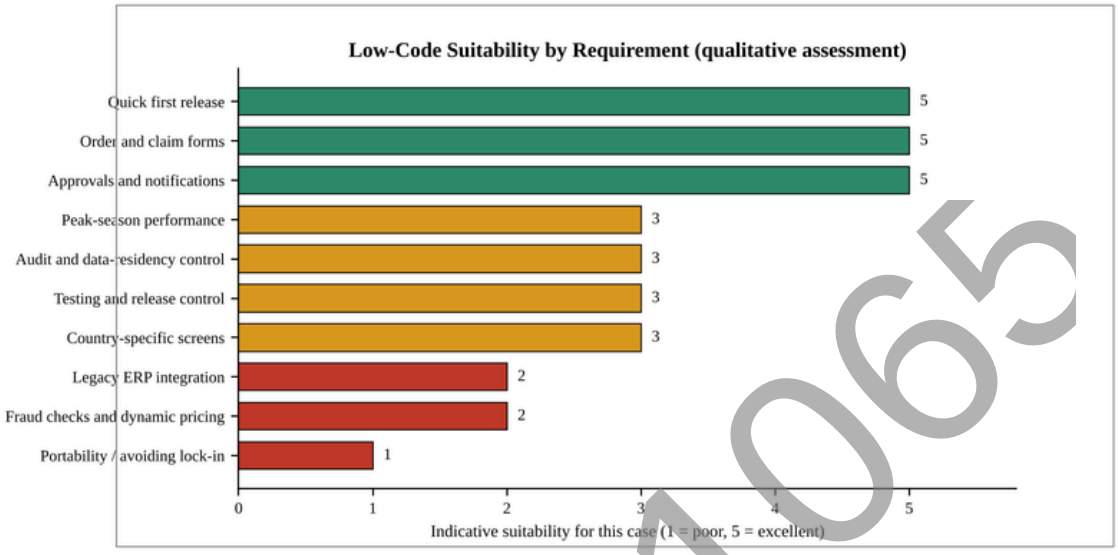
### *3.4 Where low-code starts to strain*




The curve shows a large benefit from low-code for simple needs. As complexity, transaction volume and integration demands increase, that benefit falls, while custom code stays fairly steady. The shaded hybrid zone marks where using both together is usually best. The chart is a conceptual illustration, not measured data.

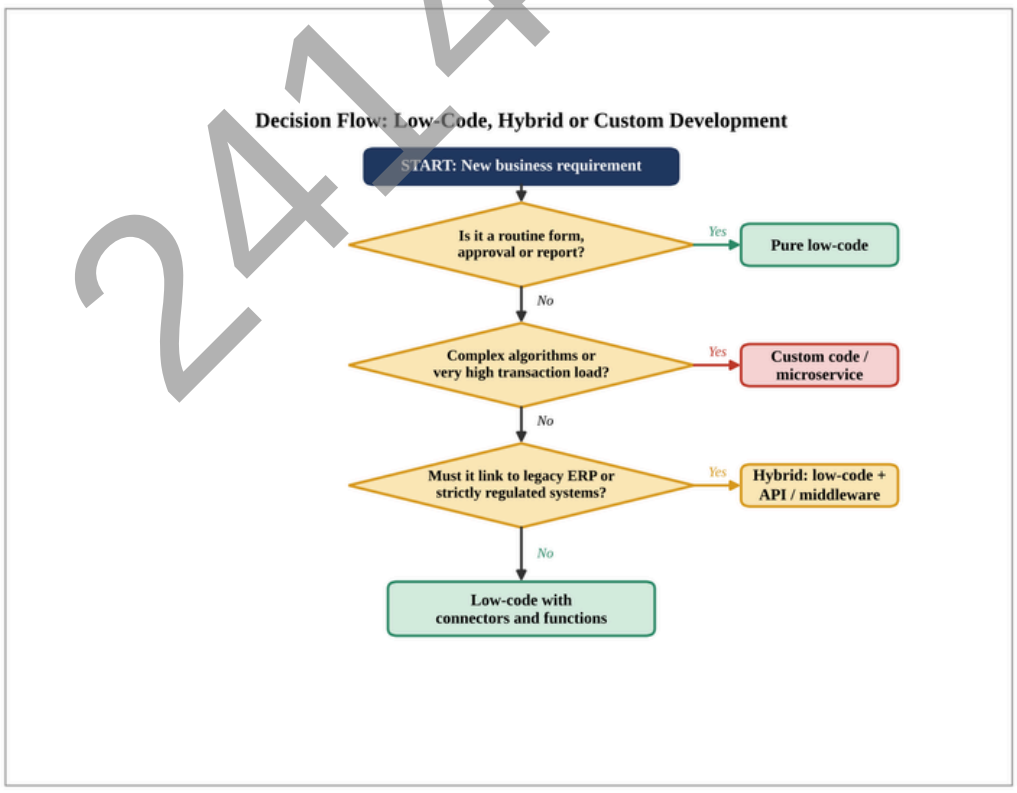
241401065

### *3.5 Suitability of low-code by requirement*




The chart rates how well low-code fits each requirement in this case. A quick first release, order and claim forms, and approvals score highest. Fraud checks, legacy ERP integration and portability score lowest. The values are indicative scores from this analysis, not benchmark results.

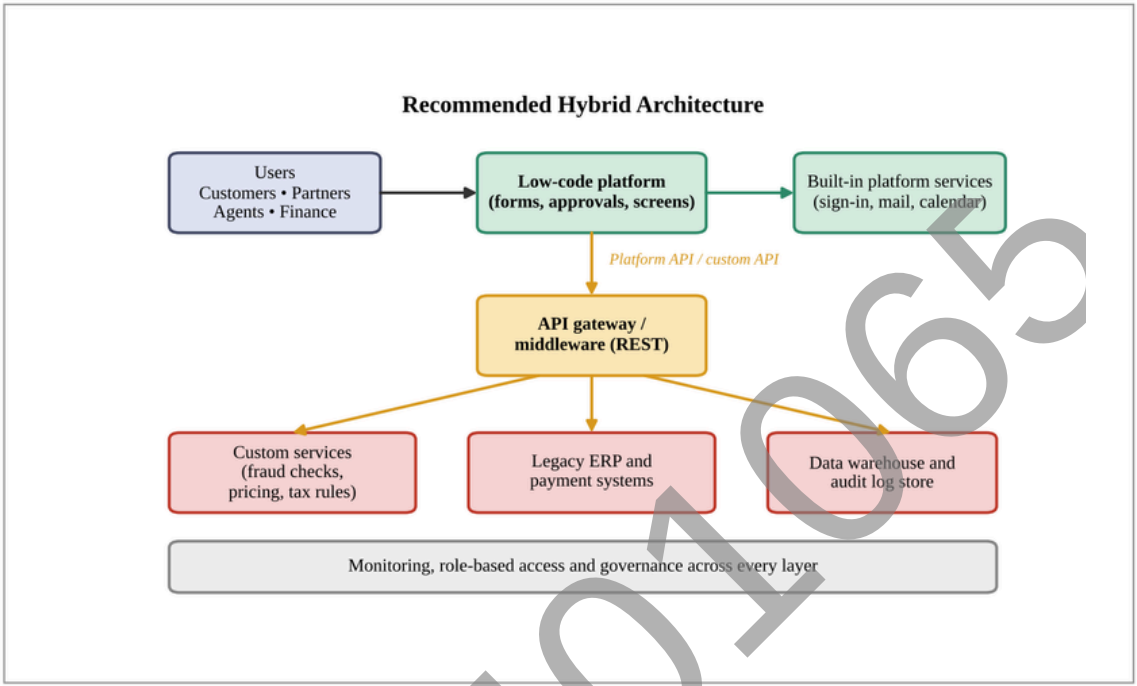
### *3.6 Decision flow for choosing the approach*




The flow asks three questions: is the need a routine form, approval or report; does it require complex algorithms or very high transaction load; and must it link to legacy or strictly regulated systems. The answers lead to pure low-code, custom code, a hybrid design, or low-code with connectors and functions.

2414010 65

### *3.7 Recommended hybrid architecture*




The low-code platform keeps the forms, approvals and screens, and relies on built-in services for sign-in and messages. An API gateway or middleware layer links it to custom services for fraud and pricing, to the legacy ERP and payment systems, and to a data warehouse and audit store. Monitoring, role-based access and governance cover every layer.

## *4. Final Result*

The case study found that low-code is very effective for the routine parts of an enterprise system: forms, approvals, notifications and ecosystem integrations. It delivers quickly with little code, whichever platform is chosen.

For a complex, global use case, the main limitations were performance at scale, complex logic, legacy and industry integration, restricted customisation, compliance and data-residency control, weak testing and change control, app sprawl and vendor lock-in. These were examined in the limitation table, the conceptual chart and the suitability chart.

The recommended solution is a hybrid approach: keep low-code for forms, workflows and screens, and add an API or middleware layer with custom services for heavy logic, legacy integration and analytics, under clear governance. No live testing or benchmark was carried out; the conclusions come from analysis of the case.

2414010065